



Name: Cohort/Batch: Date: Score:

NEXT.JS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Web Technologies

TOTAL: 10 QUESTIONS

Q1 What is Next.js and what problem in Web Technologies does it solve? **Basics**

Q6 How does Next.js handle reliability and high availability? **Reliability**

Q2 What are the core components or concepts in Next.js? **Architecture**

Q7 What observability does Next.js provide out of the box? **Lifecycle**

Q3 How does Next.js compare to its main alternatives? **Configuration**

Q8 What are common performance bottlenecks in Next.js? **Optimization**

Q4 How do you get started with Next.js for a new project? **Hardening**

Q9 What security best practices apply when running Next.js? **Testing**

Q5 What configuration options most affect Next.js's behavior in production? **Deployment**

Q10 What mistakes do teams commonly make when adopting Next.js? **Tuning**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Next.js is a tool or platform that

Mitigation

Next.js is a tool or platform that automates or simplifies a specific concern in the Web Technologies domain, reducing manual effort and operational complexity for engineering teams.

A6 Next.js provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

HA, availability

A2 Next.js has key building blocks that work

Primitives

Next.js has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Next.js exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

Observation

A3 Next.js trades off simplicity against feature richness

Hardening

Next.js trades off simplicity against feature richness differently than its alternatives. Choose Next.js when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

Performance

A4 Install Next.js via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

Gotchas

A9 Run Next.js with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

Assessment

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

Configuration

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.

Documentation